

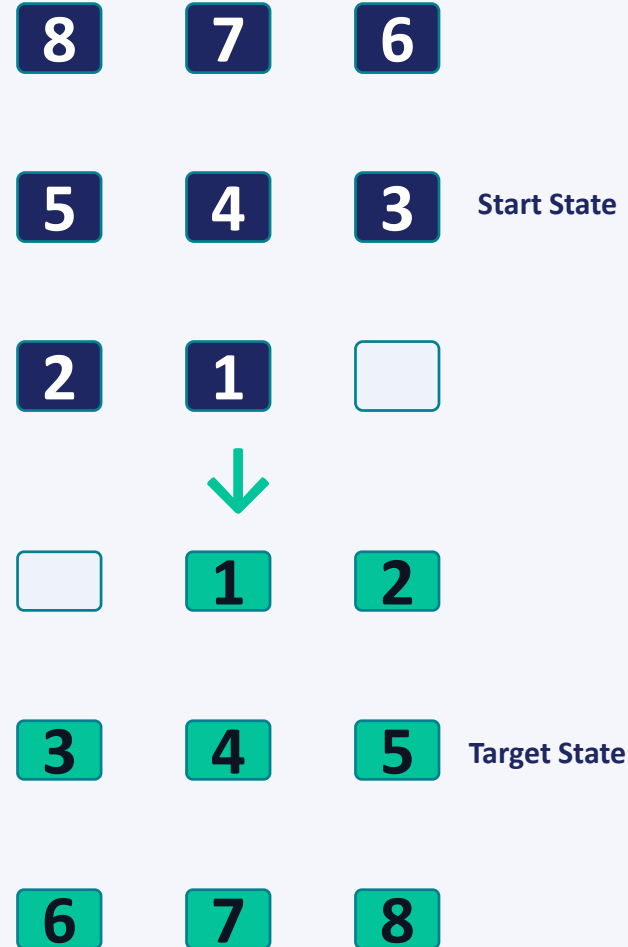
Sliding Block Puzzle Solver

BFS vs A* with Manhattan Distance Heuristic



Problem Statement

- ▶ 3×3 sliding block puzzle with 8 numbered tiles and one blank
- ▶ Tiles can only slide horizontally or vertically into the blank space
- ▶ Goal: reach a specific target configuration in as few moves as possible
- ▶ Must implement BFS (uninformed) and A* (informed with admissible heuristic)
- ▶ Track: node expansions · max nodes in memory · moves in solution



Breadth-First Search

Uninformed Search

Frontier

FIFO queue (deque) — explores all nodes at depth d before $d+1$

Completeness

Complete — will always find a solution if one exists

Optimality

Optimal for unit-cost steps — shortest path guaranteed

Time complexity

$O(b^d)$ — exponential in solution depth d

Space complexity

$O(b^d)$ — must store all discovered states

Key tradeoff

Finds optimal solution but consumes large memory

Implementation: `collections.deque` · No external search libraries used

A* Search

Informed Search with Admissible Heuristic

$$f(n) = g(n) + h(n)$$

f(n): total estimated cost of the path through n

g(n): exact cost (number of moves) from start to n

h(n): heuristic estimate from n to goal (admissible)

Frontier

Min-heap (heapq) ordered by f(n)

Completeness

Complete with admissible heuristic

Optimality

Optimal — never overestimates cost

Efficiency

Expands far fewer nodes than BFS

Implementation

heapq · No external libraries

Admissible Heuristic: Manhattan Distance

$$h(n) = \sum |row_i - row^*_i| + |col_i - col^*_i|$$

- ✓ Each move shifts one tile exactly one step (row or column)
- ✓ Manhattan distance counts the minimum moves for each tile ignoring others
- ✓ It never overestimates actual moves $\rightarrow h(n) \leq h^*(n) \rightarrow$ Admissible
- ✓ Blank tile is excluded (it is not numbered and has no fixed goal position)

Example: Tile 8

8	7	6
5	4	3
2	1	

Tile 8 is at (0,0) in start

Goal position is (2,2)

$$|0-2| + |0-2| = 2 + 2 = 4$$

Also implemented: Misplaced Tiles heuristic (weaker, fewer nodes than BFS but more than Manhattan)

Experimental Results

Assignment puzzle · python3 main.py

Algorithm	Heuristic	Expansions	Max Memory	Moves
Breadth-First Search	—	178,223	180,899	28
A* Search	Manhattan distance	174	266	28

1,024×

fewer
expansions

680×

less
memory

28

moves
(both)

Live Demo

Screenshots from the actual running application

Breadth-First Search — solved

Sliding Block Puzzle
Interactive Search Algorithm Simulation

Controls

- Solve with BFS
- Solve with A*
- Reset to Start
- Shuffle (Random Walk)
- Puzzle Solved!

Algorithm Performance

EXPANSIONS	MAX MEMORY NODES	MOVES TAKEN	TIME (S)
178223	180899	28	0.2400

A* Search — solved

Sliding Block Puzzle
Interactive Search Algorithm Simulation

Controls

- Solve with BFS
- Solve with A*
- Reset to Start
- Shuffle (Random Walk)
- Puzzle Solved!

Algorithm Performance

EXPANSIONS	MAX MEMORY NODES	MOVES TAKEN	TIME (S)
174	266	28	0.0027

Same 28-move solution, same start/target puzzle · values match Experimental Results

Discussion

Why does A* expand $\sim 1,024\times$ fewer nodes than BFS?

1

BFS is blind

Without a heuristic, BFS must explore all states at each depth level. At depth 28 the frontier can hold tens of thousands of states simultaneously.

2

Manhattan distance is tightly informed

The heuristic closely mirrors the true cost — almost every state it evaluates is on or near the optimal path, so very few dead-end branches are expanded.

3

Same optimal result

Both algorithms return a 28-move solution. A* achieves this with 174 expansions; admissibility guarantees no shorter path exists.

4

Memory advantage

A* keeps only 266 nodes in memory vs 180,899 for BFS — a direct consequence of fewer branches explored.

Summary



Implemented BFS and A* from scratch using only Python standard library



Manhattan distance is admissible $\rightarrow$ A* guaranteed to find optimal solution



A* expands 1,024 $\times$ fewer nodes and holds 680 $\times$ less memory than BFS



Both algorithms find the same 28-move optimal solution path



Informed search is dramatically more efficient for complex state spaces

Thank you · Questions?